MODULE 03 · DOMAIN 1 - DATA PREPARATION FOR ML AND AI

Formats and the streaming path

What shape the data should be in, and how it arrives when it never sits still.

MODULE 03 of 43 DOMAIN 1 - Data Prep WEIGHT 28%

admin@sthithapragna.com

You choose a format once. You pay for that choice on **every read, forever**.

Row-oriented or column-oriented?

ROW FORMATS

CSV, JSON, Avro

- All fields of one record stored together.
- Cheap to append a new record.
- Reading one column reads everything.
- Good for writes and whole-record access.

COLUMNAR FORMATS

Parquet, ORC

- All values of one column stored together.
- Read only the columns you asked for.
- Compresses far better, similar values adjacent.
- Good for analytics and feature selection.

THE TIEBREAKER

Machine learning reads a few columns from very many rows. That is the columnar case almost exactly.

Six formats, and what each is for

FORMAT	WHAT IT IS GOOD AT
Parquet	Columnar. The default for analytics and ML. Splittable, compresses well
ORC	Columnar. Similar to Parquet. Common where Hive and EMR came first
Avro	Row-based, binary, schema travels with the data. Good for streaming and schema change
CSV	Row-based text. Universal, human-readable, no types, no schema
JSON	Row-based text. Nested structures. Verbose and slow to parse at scale
RecordIO-protobuf	Binary, SageMaker AI built-in algorithms. Enables Pipe mode

ON THE EXAM

Two are columnar, three are row-based text or binary, and one exists only inside SageMaker AI.

Why Parquet keeps winning

- **Column pruning.** Ask for 6 columns of 80 and you read roughly 6 columns of data.
- **Predicate pushdown.** Row-group statistics let readers skip blocks that cannot match.
- **Strong compression**, because a column holds similar values. Often 5 to 10 times smaller.
- **Splittable**, so Spark and EMR can process one file across many workers.
- Carries a schema with types, so nobody has to guess whether a column is a string.

> ON THE EXAM
>
> Athena and Redshift Spectrum bill by data scanned. Column pruning is a direct discount on that bill.

Compression, and which codecs stay splittable

CODEC	WHAT YOU GET
Snappy	Fast, moderate ratio, splittable inside Parquet. The usual default
gzip	Better ratio, slower, NOT splittable on plain text files
bzip2	Best ratio of the text codecs, splittable, slow to decompress
zstd	Strong ratio and fast. Increasingly the sensible modern choice
Uncompressed	Only when something downstream genuinely cannot decompress

ON THE EXAM

Splittability is about plain text files. Inside Parquet, compression is per row group, so Parquet stays splittable regardless.

CSV, and what it actually costs you

- Universal. Every tool reads it, and a human can open it and see the problem.
- Accepted by **most SageMaker AI built-in algorithms**, which keeps it alive in ML.
- **No types.** Is 007 a number or a string? Everyone downstream has to decide.
- **No column pruning.** Reading one field means reading every byte of every row.
- Gzipped CSV is smaller but **not splittable**, so one file becomes one worker.

> ON THE EXAM
>
> The gzip trap: compressing CSV cuts storage and can destroy parallelism. Use bzip2 or Parquet if splitting matters.

JSON, and JSON Lines

- **JSON Lines** is one object per line. That is what streaming and logs produce.
- Handles nesting and optional fields without a rigid schema.
- Verbose. Every record repeats every key name, so files are large.
- Parsing is CPU-heavy at scale. It shows up as slow jobs, not as errors.
- Standard pattern: **land as JSON, convert to Parquet** before anyone queries it.

> ON THE EXAM
>
> JSON is a fine arrival format and a poor storage format. Converting on the way in is the normal architecture.

Avro and ORC, and when they turn up

- **Avro carries its schema with the data.** A reader never needs an external definition.
- That makes Avro strong for **schema evolution**: add a field and old readers still work.
- Row-based, so Avro suits **writes and streaming**, not analytical scans.
- **ORC** is columnar like Parquet, with strong Hive lineage. EMR shops often have it.
- Parquet against ORC rarely decides an exam question. Row against columnar always might.

> ON THE EXAM
>
> Schema changes often, written continuously: Avro. Read analytically by column: Parquet or ORC.

RecordIO-protobuf, and why it exists

- A binary format that **most SageMaker AI built-in algorithms** accept and prefer.
- Packs many records into large sequential blocks. That is what makes **Pipe mode** work.
- Directly solves the small-files problem from module 02.
- Float32 dense or sparse tensors. Efficient for numeric feature matrices.
- Not human-readable, and not much use outside SageMaker AI.

> ON THE EXAM
>
> If a question pairs a built-in algorithm with Pipe mode or a huge numeric dataset, RecordIO-protobuf is the format it wants.

What to reach for

THE SITUATION	THE FORMAT
Analytical queries over many rows, few columns	Parquet
Athena or Redshift Spectrum bill is the complaint	Parquet, and partition it
Schema changes often, written continuously	Avro
Streaming events arriving from applications	JSON Lines in, Parquet at rest
Feeding a SageMaker AI built-in algorithm at scale	RecordIO-protobuf
Small dataset, humans need to read it	CSV. It is fine

ON THE EXAM

Arrival format and storage format are allowed to differ, and usually should. Converting on ingest is normal.

Partitioning does what the format cannot

- Partitioning stores data in folders by a column, typically **date or region**.
- A query filtered on that column skips whole folders. It never reads them at all.
- Format cuts **columns** read. Partitioning cuts **rows** read. You want both.
- Over-partitioning creates millions of tiny files, which is module 02's problem again.
- Firehose can partition dynamically as it writes, so you do not repartition later.

> ON THE EXAM
>
> Parquet plus sensible partitioning is the single biggest cost reduction available in Domain 1.

Now the data that **never sits still**, and the services that catch it.

The streaming path, end to end

01

Produce

Applications, devices and logs emit events.

02

Buffer

Data Streams or Kafka holds an ordered, replayable log.

03

Process

Flink, Lambda or a consumer transforms and aggregates.

04

Land

Firehose delivers to S3, OpenSearch or a warehouse.

ON THE EXAM

Not every pipeline needs all four. Firehose alone covers produce and land, which is why it is so often the answer.

Kinesis Data Streams, and the numbers to know

- **One shard: 1 MB/s in, 1,000 records/s in, 2 MB/s out.** Memorise this.
- Retention is **24 hours by default**, extendable to 7 days, and long-term to 365 days.
- **Ordered and replayable** within a shard. Consumers can rewind and reprocess.
- Records are assigned to shards by partition key. A hot key means a hot shard.
- Multiple independent consumers can read the same stream at the same time.

> ON THE EXAM
>
> Replay is the property Firehose does not have. If a scenario mentions reprocessing, you need Data Streams.

Producers, and the one that batches for you

- **AWS SDK.** Simplest. PutRecord one at a time, or PutRecords for a batch.
- **Kinesis Producer Library.** Batches, aggregates and retries. Higher throughput, some latency.
- **Kinesis Agent.** A daemon that tails log files and ships them. No code at all.
- KPL **aggregation** packs many small records into one, which matters when you pay per record.
- ProvisionedThroughputExceeded means a hot partition key, not necessarily too few shards.

> ON THE EXAM
>
> Firehose bills per 5 KB record. Aggregating at the producer is the difference between paying once and paying fifty times.

Three modes, and how to choose

MODE	CHOOSE IT WHEN
Provisioned	Traffic is steady and predictable. Cheapest per GB. You manage shard count
On-Demand Standard	Traffic is spiky or unknown. Scales automatically. No shard management
On-Demand Advantage	High steady volume with many consumers. Launched November 2025

ON THE EXAM

On-demand starts at 4 MB/s write and 8 MB/s read, and scales far beyond that. You never size a shard.

Enhanced fan-out

- By default all consumers **share** the shard's 2 MB/s read. Five consumers, 400 KB/s each.
- Enhanced fan-out gives each consumer its **own dedicated 2 MB/s per shard**.
- Latency drops to roughly **70 milliseconds**, because it pushes rather than polls.
- Up to **20 consumers**, rising to **50 on On-Demand Advantage**.
- It costs extra per consumer-shard-hour. Do not enable it by reflex.

> ON THE EXAM
>
> Several teams reading one stream and complaining about throughput is the enhanced fan-out question, almost word for word.

Amazon Data Firehose, formerly Kinesis Data Firehose

- **No shards, no consumers, no code.** You point it at a destination and it delivers.
- Buffers to **128 MB or 900 seconds** for S3, whichever comes first.
- **Converts to Parquet or ORC** in flight, and partitions dynamically as it writes.
- **No retention and no replay.** Delivered means gone from Firehose.
- Billed per 5 KB record, so batch tiny records at the producer or overpay.

> ON THE EXAM
>
> 'Get these events into S3 as Parquet with least operational overhead' is the Firehose question. It appears constantly.

Destinations, including the recent ones

DESTINATION	NOTE
Amazon S3	The common one. Format conversion and dynamic partitioning here
Apache Iceberg tables	Delivers into managed table format, not just files
Amazon S3 Tables	Managed Iceberg tables. Recent, and absent from older material
Amazon Redshift	Stages through S3, then COPY into the warehouse
Amazon OpenSearch Service	For search and log analytics
Snowflake and HTTP endpoints	Third-party and custom destinations

ON THE EXAM

Undeliverable records go to an S3 backup bucket rather than being lost. That backup bucket is the retry story.

Data Streams or Data Firehose?

DATA STREAMS, WHEN

You need the log

- Several consumers read the same events.
- You may need to replay or reprocess.
- Sub-second latency matters.
- Custom processing logic per event.

FIREHOSE, WHEN

You need delivery

- One destination, and nobody replays.
- Buffered delivery in seconds is fine.
- You want format conversion included.
- Least operational overhead is stated.

THE TIEBREAKER

Both can be right together. Data Streams in front of Firehose gives you replay and managed delivery at once.

Amazon Managed Service for Apache Flink

- Formerly **Kinesis Data Analytics for Apache Flink**. Same service, new name.
- Real stream processing: **windowing, joins, aggregations** across time.
- Keeps **state** across events, which Lambda cannot do without an external store.
- Written in Java, Scala, Python or SQL against the Flink APIs.
- **Kinesis Data Analytics for SQL was retired in January 2026.** It is gone.

> ON THE EXAM
>
> Aggregating over a time window, or joining two streams, is the Flink signal. Lambda cannot hold that state.

What your older notes call these

IF YOU LEARNED IT AS	IT IS NOW
Kinesis Data Firehose	Amazon Data Firehose
Kinesis Data Analytics for Apache Flink	Amazon Managed Service for Apache Flink
Kinesis Data Analytics for SQL	Retired, January 2026. No successor of the same shape
Amazon SageMaker (the ML service)	Amazon SageMaker AI
Amazon QuickSight	Amazon Quick

ON THE EXAM

The exam may use either the old or the new name. Recognise both, and never assume a renamed service is a different one.

Apache Kafka, and Amazon MSK

- Kafka is the open-source equivalent of Data Streams. Topics and partitions, not shards.
- **Amazon MSK** runs managed Kafka. MSK Serverless removes capacity planning too.
- The case is **portability and existing investment**: teams already running Kafka.
- Firehose can read from MSK as a source, so the delivery pattern still works.
- Choosing Kafka fresh, with no existing Kafka, is rarely the exam's answer.

> ON THE EXAM
>
> The exam guide names Apache Kafka as streaming knowledge. Treat it as the 'we already run Kafka' answer.

Kinesis Video Streams

- Ingests video and time-encoded media from cameras and devices, durably and in order.
- Indexed by time, so consumers can request a fragment by timestamp.
- The ML case: feed frames to **Rekognition Video**, or to a custom vision model.
- It is in scope for this exam, listed under Media in the services list.
- Do not confuse it with Data Streams. Different service, different API, different purpose.

> ON THE EXAM
>
> Cameras plus a vision model is the only shape that points here. Anything else streaming is Data Streams or Firehose.

What to reach for

THE SITUATION	THE SERVICE
Land events in S3 as Parquet, least overhead	Amazon Data Firehose
Several teams consume the same events	Kinesis Data Streams
Must reprocess after a bug is found	Data Streams with extended retention
Rolling windows, joins, aggregations	Managed Service for Apache Flink
Consumers starving each other on read	Enhanced fan-out
Team already runs Kafka everywhere	Amazon MSK
Video from cameras for a vision model	Kinesis Video Streams

ON THE EXAM

Replay, multiple consumers and stateful processing are the three signals that rule Firehose out.

What streaming actually feeds in an ML system

- **Training data collection.** Events land in S3 and become tomorrow's training set.
- **Online features.** Aggregates computed by Flink and written to the Feature Store.
- **Real-time inference input.** An event triggers a prediction as it arrives.
- **Monitoring.** Captured inference data streams back for drift detection.
- Most streaming questions in this exam are the first one wearing different clothes.

> ON THE EXAM
>
> Streaming ingestion does not make a model real-time. Those are separate decisions, and questions conflate them deliberately.

Failure handling in a streaming pipeline

- **Firehose:** failed deliveries retry, then land in an S3 backup bucket. Nothing is dropped silently.
- **Lambda consumers:** a poison record can block a shard. Configure a dead letter queue.
- A single bad record retried forever stalls everything behind it on that shard.
- Set **bisect on function error** and a retry limit so one record cannot halt the stream.
- Duplicates happen. Kinesis is at-least-once, so consumers should be idempotent.

> ON THE EXAM
>
> At-least-once delivery means your consumer will see the same record twice eventually. Design for it rather than hoping.

Stream because the **data arrives continuously**, not because the model feels like it should be fast.

A data science team stores twelve terabytes of historical transactions in Amazon S3 as gzipped CSV, in a single flat prefix. They query it with Amazon Athena to build training sets.

A typical query selects six of the eighty columns and filters to one month. Athena costs have become the team's largest analytics line item, and queries take several minutes.

What change will reduce both cost and query time the most?

A Raise the Athena workgroup data scanned limit so queries stop being throttled

B Load the data into Amazon Redshift and query it there instead

C Recompress the CSV files with bzip2 so they become splittable

D Convert the data to Parquet and partition it by month

◆ PAUSE AND DECIDE

What change will reduce both cost and query time the most?

A. Raise the Athena workgroup data scanned limit so queries stop being throttled

That limit is a guardrail against runaway spend, not a performance control. Raising it lets the bill grow, and changes nothing else.

B. Load the data into Amazon Redshift and query it there instead

Would be faster, but it adds a warehouse to run and pay for. It treats a data layout problem as a compute problem.

C. Recompress the CSV files with bzip2 so they become splittable

A genuine improvement to parallelism, and it does nothing for the real issue. Athena still scans all eighty columns and all twelve months.

D. Convert the data to Parquet and partition it by month

Parquet cuts columns read from 80 to 6. Partitioning cuts months read from all of them to one. Both act directly on data scanned.

THE SIGNAL

Two facts were planted: six columns of eighty, and one month. One points at the format, the other at partitioning. The answer must do both.

A retailer wants clickstream events from its website landed in Amazon S3 as Parquet, partitioned by date, so they can be used to build training sets each week.

There is one destination and no requirement to reprocess events. A delay of up to a few minutes before data appears is acceptable. The team is small and wants the LEAST operational overhead.

Which design fits best?

A Kinesis Data Streams with a Lambda consumer that converts and writes to S3

B Amazon Data Firehose with record format conversion and dynamic partitioning

C Amazon MSK with a Kafka Connect S3 sink connector

D Managed Service for Apache Flink writing Parquet files to S3

◆ PAUSE AND DECIDE

Which design fits best?

A Kinesis Data Streams with a Lambda consumer that converts and writes to S3

Works, and you now own consumer code, shard sizing, error handling and small-file batching. Firehose does all of that natively.

B Amazon Data Firehose with record format conversion and dynamic partitioning

One destination, no replay, minutes of latency acceptable. Format conversion and partitioning are built-in features, not code you write.

C Amazon MSK with a Kafka Connect S3 sink connector

Most operational overhead of the four, and nothing in the scenario says the team already runs Kafka.

D Managed Service for Apache Flink writing Parquet files to S3

Flink earns its place with windowing, joins or stateful aggregation. None is required here, so it is a processing engine doing plain delivery.

THE SIGNAL

Three constraints, all planted: one destination, no replay, latency of minutes is fine. Each removes the properties Data Streams exists to provide.

A payments company streams transaction events. A fraud team consumes them for real-time scoring, and an analytics team consumes the same events for reporting. Both report that reads are slow and they appear to be competing.

Last month a bug in the fraud consumer corrupted three days of scores, and the team could not reprocess the original events to fix them.

Which combination addresses both problems?

A Kinesis Data Streams with extended retention and enhanced fan-out for both consumers

B Amazon Data Firehose delivering to two separate S3 prefixes, one per team

C Two separate Kinesis Data Streams, each written to by the producer application

D Kinesis Data Streams with the shard count doubled and default retention

◆ PAUSE AND DECIDE

Which combination addresses both problems?

A Kinesis Data Streams with extended retention and enhanced fan-out for both consumers

Enhanced fan-out gives each consumer its own dedicated 2 MB/s per shard. Extended retention keeps events long enough to replay three days.

B Amazon Data Firehose delivering to two separate S3 prefixes, one per team

Firehose has no retention and no replay, so the second problem is untouched. It also removes the real-time path the fraud team needs.

C Two separate Kinesis Data Streams, each written to by the producer application

Fixes contention by doubling ingest cost and adding a dual-write the producer must keep consistent. Enhanced fan-out solves it without either.

D Kinesis Data Streams with the shard count doubled and default retention

More shards means more aggregate throughput, still shared between consumers. And 24-hour retention cannot replay three days.

THE SIGNAL

Two problems, two features. Contention is enhanced fan-out. Reprocessing is retention. An option must fix both to be correct.

A logistics company retrains a delivery-time model once a month from data currently exported nightly from an operational database to Amazon S3 as Parquet. The model serves batch predictions each night.

A new architect proposes replacing the nightly export with Kinesis Data Streams and Managed Service for Apache Flink, describing the current design as outdated.

How should you assess the proposal?

A. Adopt it, because streaming ingestion keeps training data continuously fresh

B. Adopt it, but use Firehose instead of Flink to reduce operational overhead

C. Reject it. The data is consumed monthly, so streaming adds cost and operations for no benefit

D. Adopt it only for the inference path, keeping the nightly export for training

◆ PAUSE AND DECIDE

How should you assess the proposal?

A. Adopt it, because streaming ingestion keeps training data continuously fresh

Freshness has no value here. The model reads the data once a month, so data arriving seconds after the event sits unused for weeks.

B. Adopt it, but use Firehose instead of Flink to reduce operational overhead

A cheaper way to build something unnecessary. Nightly export to Parquet already lands the data correctly.

C. Reject it. The data is consumed monthly, so streaming adds cost and operations for no benefit

Consumption cadence, not fashion, decides ingestion architecture. Nothing in the requirement changes if events arrive sooner.

D. Adopt it only for the inference path, keeping the nightly export for training

Thoughtful, but inference is already batch and runs nightly. Streaming input to a nightly batch job changes nothing downstream.

THE SIGNAL

Ask what the data does, not what the architecture sounds like. Nothing here consumes data faster than once a night.

What to take from this module

- **Parquet plus partitioning** is the biggest cost lever in Domain 1. Format cuts columns, partitions cut rows.
- **Replay, many consumers, stateful processing.** Any one of them rules Firehose out.
- **One shard: 1 MB/s in, 2 MB/s out.** Consumers share that 2 MB/s unless you fan out.
- **Names changed.** Amazon Data Firehose, Managed Service for Apache Flink, and no more KDA for SQL.
- **Stream for the data, not for the model.** Monthly consumption does not need a stream.

> ON THE EXAM
>
> Module 04 covers the engines that transform this data: Glue, EMR, Processing jobs, DataBrew and Data Wrangler.